WEEK 2 · LESSON 12 OF 24 · HANDOUT

Week 2 Project: A Neural Network for Loan Risk

Study time: **about 90 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=V_xro1bcAuA — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191: Introduction to Deep Learning (2024) — https://www.youtube.com/watch?v=ErnWZxJovaM

🎓 **Go deeper:** EuroPython 2023: PyTorch 2.0 — why should you care? — https://www.youtube.com/watch?v=B_KXmhCffzQ

Your goal in this lesson: Build a complete, well-trained neural network for a real business problem — with a validation set, early stopping and a fair baseline — and report honestly whether deep learning was worth it.

The brief

The SACCO from Machine Learning Fundamentals heard that "AI with neural networks" beats everything. Your job: test the claim fairly on its micro-loan data and recommend what to use.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

Part 1 — Prepare (train / validation / test)

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import copy
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score
from torch.utils.data import TensorDataset, DataLoader

loans["loan_to_income"] = loans["loan_rwf"] / loans["income_rwf"]
X = loans.drop(columns="defaulted").to_numpy()
y = loans["defaulted"].to_numpy()
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.25, random_state=42,
stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.2, random_state=42,
stratify=y_dev)
scaler = StandardScaler().fit(X_train)
X_train, X_val, X_test = (scaler.transform(a) for a in (X_train, X_val, X_test))
t = lambda a: torch.tensor(a, dtype=torch.float32)
train_loader = DataLoader(TensorDataset(t(X_train), t(y_train)), batch_size=64, shuffle=True)
```

Part 2 — The network, with dropout and batch norm

```
class LoanNet(nn.Module):
    def __init__(self, n_features, hidden=64, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, hidden), nn.BatchNorm1d(hidden), nn.ReLU(),
nn.Dropout(dropout),
            nn.Linear(hidden, hidden), nn.BatchNorm1d(hidden), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(hidden, 1))

    def forward(self, x):
        return self.net(x).squeeze(1)
```

Part 3 — Train with early stopping on validation AUC

```
def val_auc(model):
    model.eval()
    with torch.no_grad():
        return roc_auc_score(y_val, model(t(X_val)).numpy())

torch.manual_seed(0)
model = LoanNet(X_train.shape[1])
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_auc, best_state, waited, history = 0, None, 0, []
for epoch in range(200):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()
        F.binary_cross_entropy_with_logits(model(xb), yb).backward()
        opt.step()
    auc = val_auc(model)
    history.append(auc)
    if auc > best_auc:
        best_auc, best_state, waited = auc, copy.deepcopy(model.state_dict()), 0
    else:
        waited += 1
        if waited >= 20:
            break
model.load_state_dict(best_state)
print(f"stopped after {len(history)} epochs, best validation AUC {best_auc:.3f}")
```

Part 4 — The fair comparison

Both models see exactly the same training rows and are judged once on the same test rows:

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression().fit(X_train, y_train)
model.eval()
with torch.no_grad():
    nn_scores = model(t(X_test)).numpy()
print("Neural network test AUC:     ", round(roc_auc_score(y_test, nn_scores), 3))
print("Logistic regression test AUC:", round(roc_auc_score(y_test, logreg.predict_proba(X_test)
[:, 1]), 3))
```

Expect the two to be **very close** (both around 0.86–0.88). That is the honest result: the loans were created by a fairly simple rule, and logistic regression already captures it. The network adds complexity — thousands of weights, more settings, harder explanations — without a clear gain.

Part 5 — Recommendation (one page)

1. The question and the data.
2. The network: layers, dropout, batch norm, optimiser, early stopping (with the learning curve).
3. A table of test AUC for both models.
4. Your recommendation, with reasons: accuracy, simplicity, explainability to borrowers and regulators, cost of running.
5. When the SACCO **should** consider deep learning (for example: millions of transactions, text from customer messages, or images of documents).

Marking guide (self-check)

Area	Excellent work
Method	Train / validation / test split; scaler fitted on training rows
Training	Early stopping on validation with best weights restored
Fairness	Same data and test set for both models; test used once
Judgement	Honest conclusion, even if the network does not win
Communication	Clear recommendation a board can act on

Remember this: A good engineer uses deep learning where it adds value — and proves it with a fair baseline instead of assuming it.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–5. Then remove `loan_to_income` and retrain both models. Which one suffers more without the engineered feature, and why might that be?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why does the project use a separate validation set?
   - A. Because PyTorch requires three sets
   - B. To increase AUC
   - C. To train faster
   - D. To choose when to stop training without touching the test set
2. What is restored when early stopping ends?
   - A. Random weights
   - B. The weights with the best validation AUC
   - C. The first weights
   - D. The logistic regression
3. Which layer types help this network generalise?
   - A. Embedding only
   - B. Convolution and LSTM
   - C. Dropout and batch normalisation
   - D. Softmax and pooling
4. The network and logistic regression both score about 0.87 AUC. What is the best recommendation?
   - A. Average them blindly
   - B. Use neither
   - C. Prefer the simpler logistic regression unless the network clearly wins
   - D. Always use the network because it is AI
5. When would deep learning be worth trying for the SACCO?
   - A. Only with 50 rows
   - B. Never
   - C. Only for printing reports
   - D. With very large data or images and text, such as customer messages
6. Why are both models judged on the same test rows?
   - A. Because the scaler needs it
   - B. It does not matter
   - C. To save memory
   - D. So the comparison is fair

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.